Encoder Decoder

+

Attention

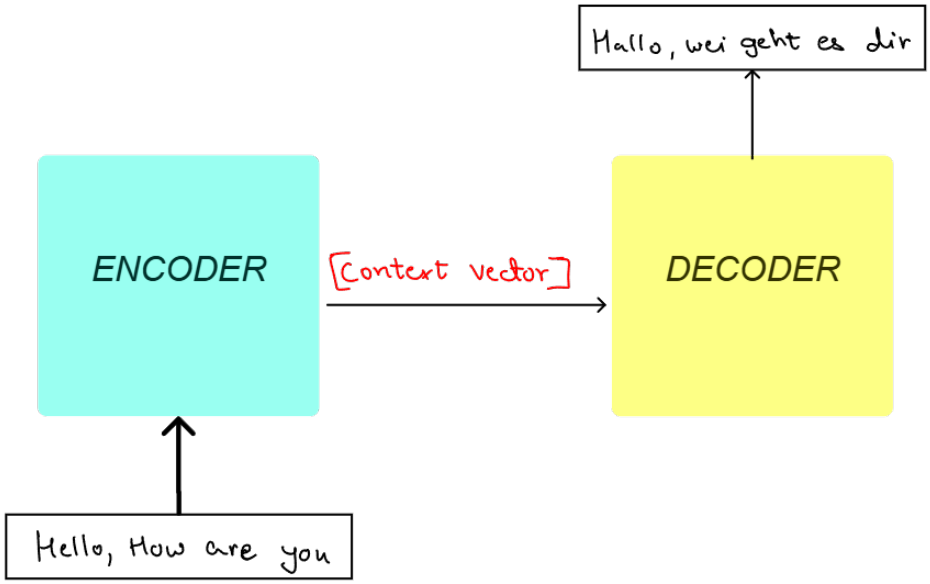
Encoder Decoder

Encoder decoder architecture transforms input sequences (text, images) into a compact latent representation via the encoder, then generates output sequences autoregressively via the decoder.

Core Idea:

Encoder : Takes input 'x' (sentence, image, audio, etc) and turns it into a fixed size or structured latent representation 'z' that "summarizes" the important information.

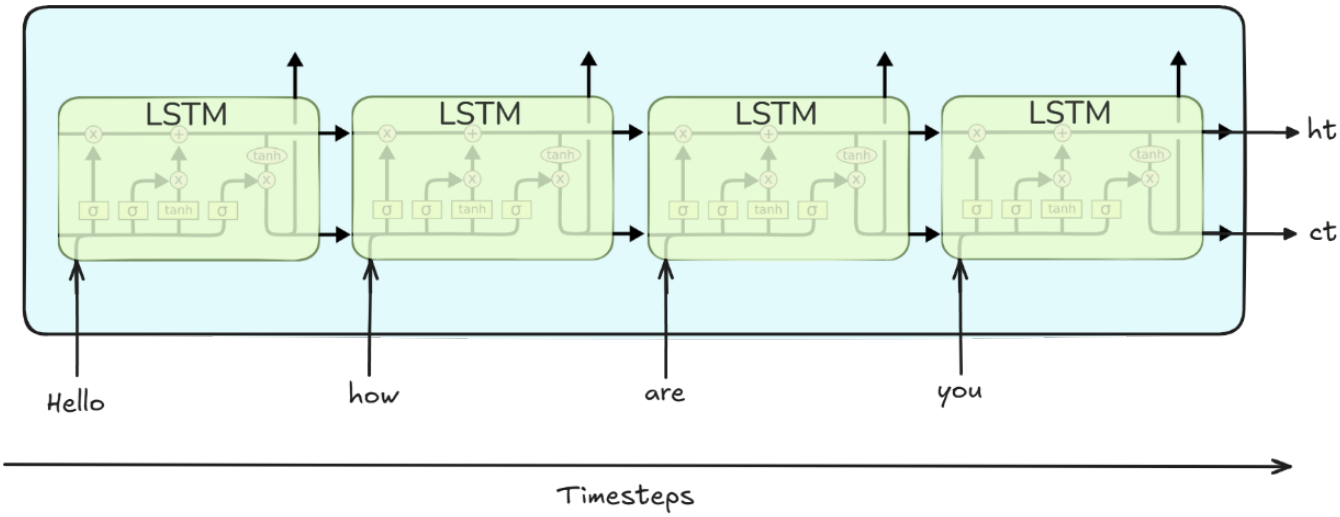
Decoder : Starts from 'z' (and often previous outputs) and generates the target output 'y' step by step (sequence) or as a reconstructed object (image, text, etc)



So, in simple words, encoder processes the full input and produces hidden states.

The decoder uses those hidden states to autoregressively generate the output, predicting one token at a time while feeding its previous output back in.

Encoder

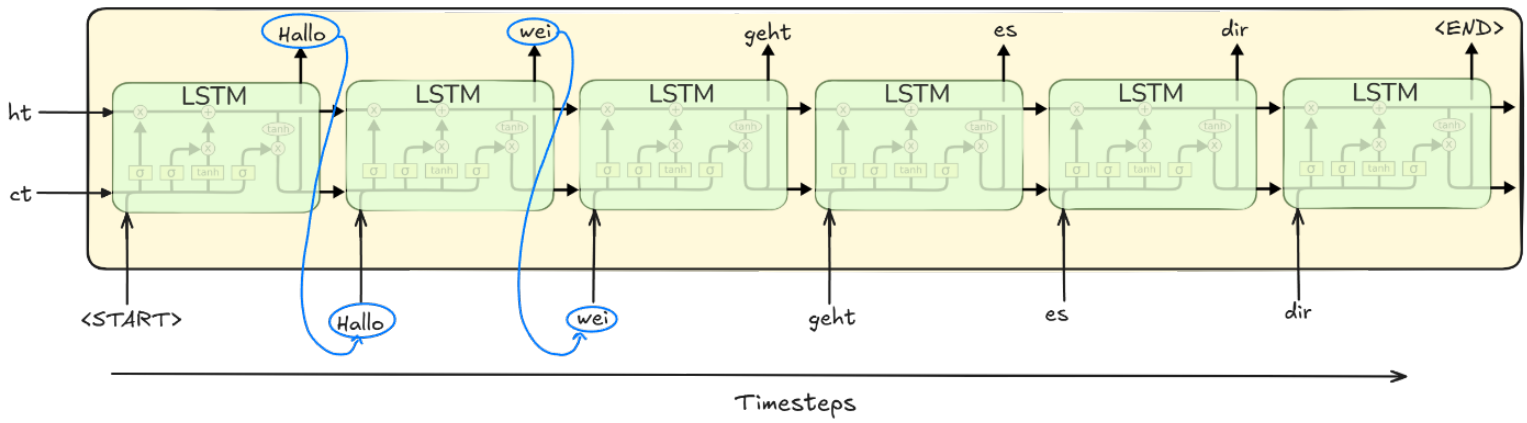


The encoder is simply one LSTM cell unfolded over time. Each word is processed one by one and the final output is a context vector.

Therefore, a encoder when forward feeded returns a context vector.

Context vector contains pair of hidden state and the cell state

Decoder



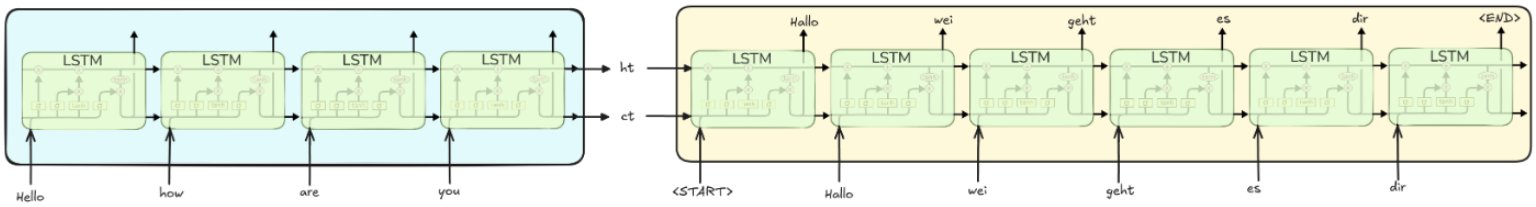
The decoder simply generates the output sequence one token at a time, conditioned on the context vector.

The decoder initially recives the context vector (h_t, c_t) as the input and after that each LSTM cell recives the input from previous LSTM cell but the X_t input will be the output from the past cell.

<Start> and <END> are simply tokens that we need in the output.

In the original research paper, <SOS> : start (start of sentence)
<EOS> : End. (end of sentence)

Training the architecture.



Here, the dataset we use is machine translated dataset. It's simply translated version of words / sentences together. The dataset looks like this:

Input_English	Output_German
Hello	Hallo
I am good	Ich bin gut
good Morning	Guten Morgen
I need a glass	Ich mochte ein tasse
The Dog!	die Hung!
Thank you	Danke

Now lets do the training from scratch. for that we will take a small dataset of two rows.

english	German
good Morning	guten Morgen
How are you	Wei geht es dir

Encoding (OHE)
using OHE
for simplicity

english	German
good : [1, 0, 0, 0, 0]	<SOS> : [1, 0, 0, 0, 0, 0, 0, 0]
Morning : [0, 1, 0, 0, 0]	<EOS> : [0, 1, 0, 0, 0, 0, 0, 0]
How : [0, 0, 1, 0, 0]	Guten : [0, 0, 1, 0, 0, 0, 0, 0]
are : [0, 0, 0, 1, 0]	Morgen : [0, 0, 0, 1, 0, 0, 0, 0]
you : [0, 0, 0, 0, 1]	Wei : [0, 0, 0, 0, 1, 0, 0, 0]
	geht : [0, 0, 0, 0, 0, 1, 0, 0]
	es : [0, 0, 0, 0, 0, 0, 1, 0]
	dir : [0, 0, 0, 0, 0, 0, 0, 1]

Encoder Forward Pass

Passing the row 1: good Morning $\longrightarrow$ Guten morgen

In the encoder part, we will simply pass the inputs one by one to get the context vector (h_t, c_t).

Note: Here in timestamp 1, we send <sos> as the first input and <Eos> at timestamp t when the sentence has ended. But for simplicity, here i am just sending the values only.

Timestep 1 : $x_1 \longrightarrow \{good\}$: $[1, 0, 0, 0, 0]$

all the values of hidden state and cell state are calculated using forget, input and output gates.

We get (for ex.):

$$C_1 = [0.35, -0.06, 0.72]$$

$$h_1 = [0.20, -0.05, 0.31]$$

Timestamp 2 : $x_2 \longrightarrow \{Morning\}$: $[0, 1, 0, 0, 0]$

$$C_1 = [0.13, 0.57, 0.54]$$

$$h_1 = [0.09, 0.31, 0.38]$$

Technically it should be

$t_1 =$ <sos>

$t_2 =$ <good>

$t_3 =$ <morning>

$t_4 =$ <Eos>

Once the sentence is completed, we finally get the context vector as output

$$\text{Context} = (h_4, C_4)$$

$$= ([0.17, -0.10, 0.28], [0.34, -0.15, 0.98])$$

This context vector will be the input for the decoder with

(decoder) $S_0 = h_4 = [0.17, -0.10, 0.28]$

(decoder) $C_0 = C_4 = [0.34, -0.15, 0.98]$

Decoder Forward Pass

Decoder has its own seperate weights.

Firstly at timestep $t=0$, we have a total of two inputs. One is the context vector and second is the <start> token as x_0

from training data

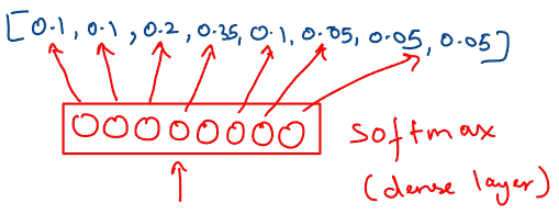
[1, 0, 0, 0, 0, 0, 0, 0]

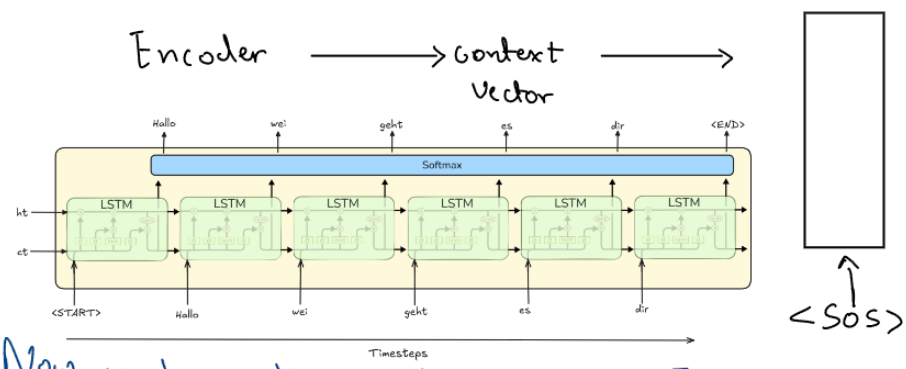
Here, when we feed the first input from training data's output, <sos>, it (the LSTM cell) will give us an output which will be used as the input for next cell.

But here, the output will from each cell will be determined by a softmax layer (Because this is multiclass classification).

So, it goes like: we give inputs (context vector, x_0(<sos>)), and then the output from LSTM cell will be $O_0(h_t)$ and it will again go through a softmax layer which will give us the probabilities for all classes and the we will get the single output.

Note that the softmax outputs through a dense layer.





Now we have the vector at $t=0$: [0.1, 0.1, 0.2, 0.35, 0.05, 0.05, 0.05], we now find the highest probability class and take it as input for next cell. Here, 0.35 is highest : 4th index

So, our output is 4th index : [0, 0, 0, 1, 0, 0, 0, 0] : Morgen.

Now, our output at $t=0$ is $[0,0,0,1,0,0,0,0]$ (Morgen),

but if we see carefully, our correct answer should be "gut" $[0,0,1,0,0,0,0,0]$ but our predicted answer here is "Morgen" $[0,0,0,1,0,0,0,0]$.

Now, when we move to next timestep $t=1$, our input will be the correct answer and not the predicted one.

So, input for $t=1$: $x_1 = [0,0,0,1,0,0,0,0]$ (Morgen).

This concept is called "Teacher Forcing".

Teacher Forcing : During training, at each decoder timestep, instead of feeding the model's own previous prediction as input, you feed the actual ground truth token.

The reason we use teacher forcing is because if we do not use it, when the first cell outputs / predicts the wrong answer, all the next sequence following will be incorrect. So we apply teacher forcing and by doing so, the model learns the correct mappings.

We continue this process for next timestep $t=1$ until we reach the end.

Once we are at the end, we calculate the total loss using Categorical cross entropy cross entropy. (we can also take average)

$$\boxed{L = -\sum_{i=1}^{n} Y_i^{true} \log(Y_i^{Pred})}$$

Now that we have the loss value, we can now apply back propagation.

Back propagation essentially has 2 steps: calculate gradients $\longrightarrow$ update weights

Pipeline :

initial input $\longrightarrow$ forward prop $\longrightarrow$ output (pred + groundtruth) $\longrightarrow$ loss $\longrightarrow$ gradient $\longrightarrow$ weight update $\longrightarrow$ loop

Now for prediction, everything's the same except that we do not use Teacher forcing while predicting. We provide the same output from last cell as input in next cell in decoder.

Improvements :

1) We used OHE here, but usually we do not use OHE because if we have a large datasets, the vectors will be immensely large. Instead we use Embeddings layer. Embeddings are simply dense low dimensional vector that represents a token.

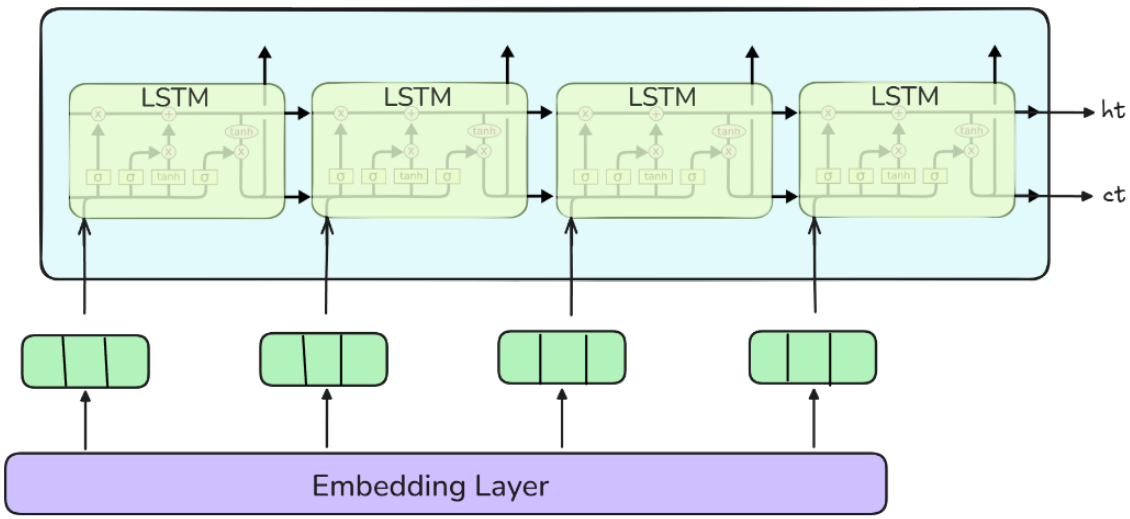
Ex. [0.3, 0.6, 0.1] → "Morgen"

The main difference here is that embeddings are learned during trainings while OHE cannot learn anything.

This means during training, the model learns to place semantically similar words close together in embedding space.

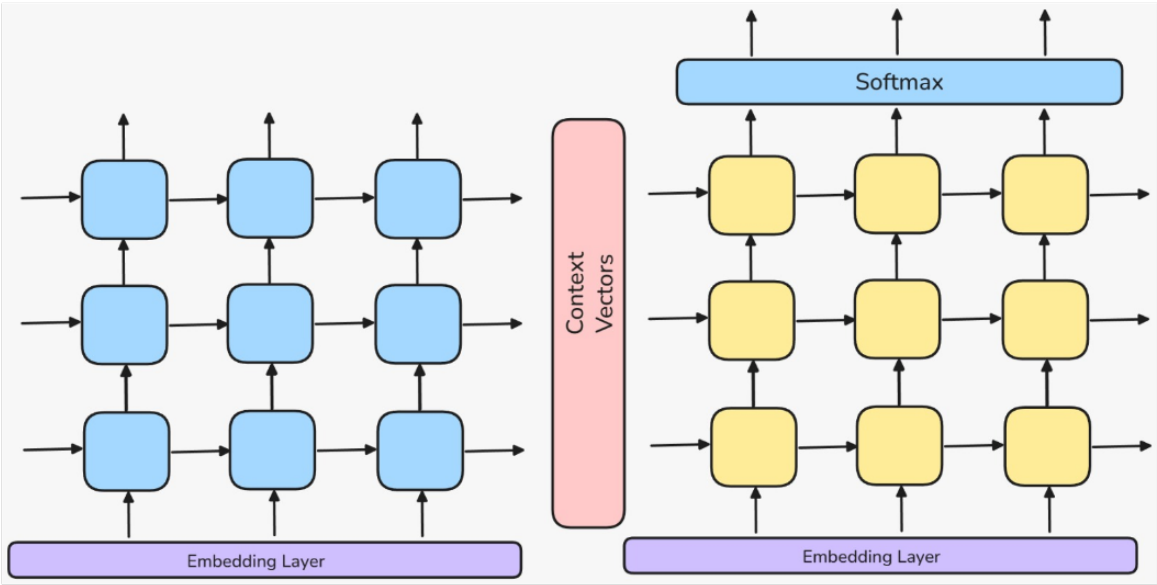
ex. "good" and "great" end up close because they appear in similar context.

There are 2 seperate embedding matrices. One for english, one for german. Both are independent and learned seperately.



2. Using Deep LSTM's

Instead of using only one layer of LSTM, we can use multiple layers of LSTM which will improve our model.



3. Reversing the input

Simply giving the input as Morgen guten instead of the correct way.

This is only used for some languages that has a meaning with similar words.

Attention Mechanism

lets firstly understand the problem with vanilla encoder decoder architecture.

Problem 1: Information Bottleneck

The encoder compresses the entire input sequence into a single fixed-dimensional vector $c = h_t$. As input length increases since the vector dimensions are static, it needs to overwrite the old history which in turn makes the model forget the very old values.

Even though we have LSTM cells, it still does not help because the network is still forced to make lossy compression decisions. Early token information gets progressively diluted as more tokens are processed.

This is also visible from a gradient perspective – during BPTT, the gradient of the loss w.r.t early encoder hidden states passes through many timesteps, and even with gating, attenuates. (reduced force)

So, the model has both a forward problem (info gets overwritten) and a backward problem (gradients for early tokens are weaker, so the model learns less about how to encode them well.

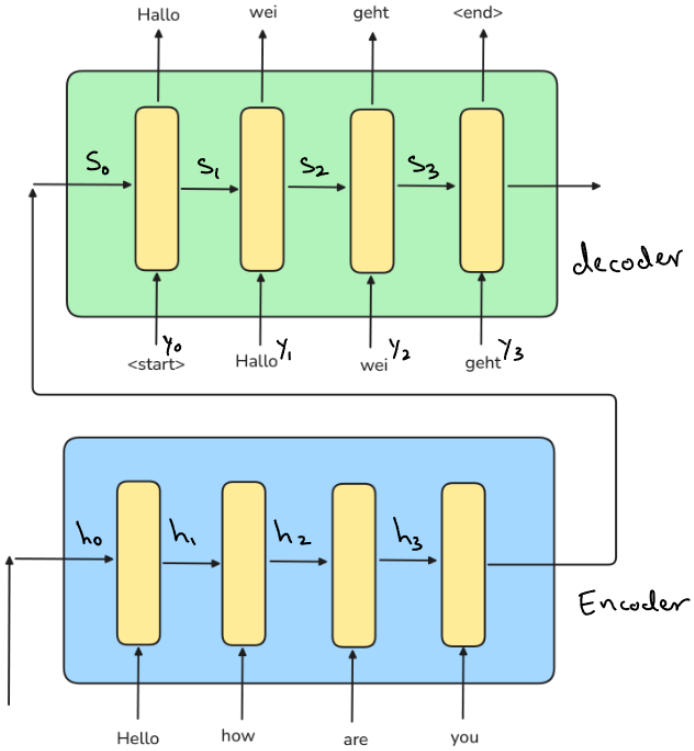
② Static Context Vector

In vanilla seq2seq using LSTM's, the encoder LSTM produces hidden states $h_1, h_2, \ldots h_T$ at every timestep, each capturing contextual information about the position. But only the final h_T is passed to the decoder as the context vector c – all intermediate hidden states are thrown away.

This c is fixed across all decoder timesteps. But the decoder LSTM needs different input information at different time steps. – when generating the hindi verb, it needs the english verb position; when generating the hindi subject, it needs the english subject position. The decoder has no mechanism to go back and look at specific encoder hidden states.

The useful per-position information already exists in $h_1, h_2, \ldots h_T$, we are just not using it.

Now that we are aware of the major problems in the vanilla encoder decoder architecture, this is where we introduce a new concept called "Attention Mechanism".



As we can see here, in order for the model to generate the first word 'Hallo', it needs the word 'Hello' from encoder. Same goes for lets say, for "wei" it needs "how are" both from encoder.

So, by introducing attention mechanism, we are simply providing the decoder's LSTM cell the particular inputs from encoder that will be helpful to make the decision.

By this, the decoder is simply able to see what's happening at that current timestep in the encoder.

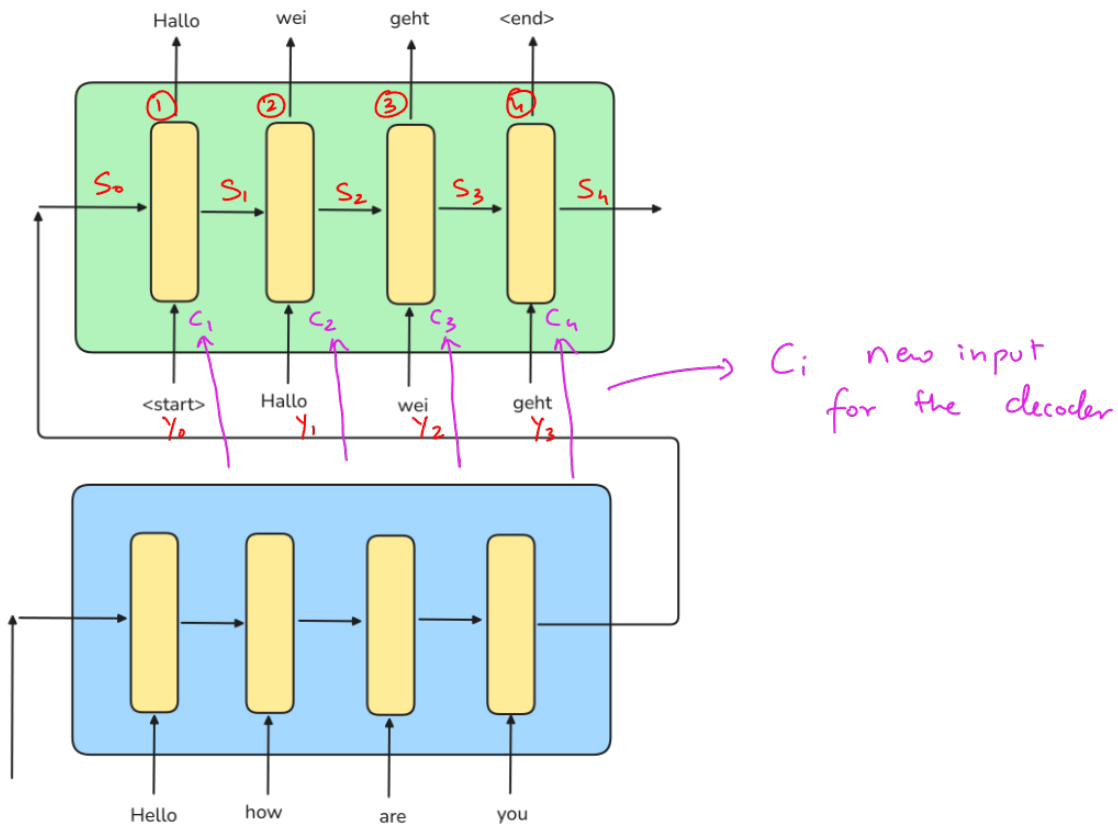
So, in a vanilla encoder decoder architecture, the total inputs at timestep t is total of two inputs.

$(t=1)$ One is the S_1 (S_{T-1}) and second is the Y_1

So, each cell has two inputs, S and Y

But in attention Mechanism, we pass one more information from the encoder. The information here will consist of what timestep is useful for the decoder at the current timestamp t.

The new third input is let's say $C_1, C_2 \dots C_T$



Therefore, Now there are a total of 3 inputs at each timestep (t or i)

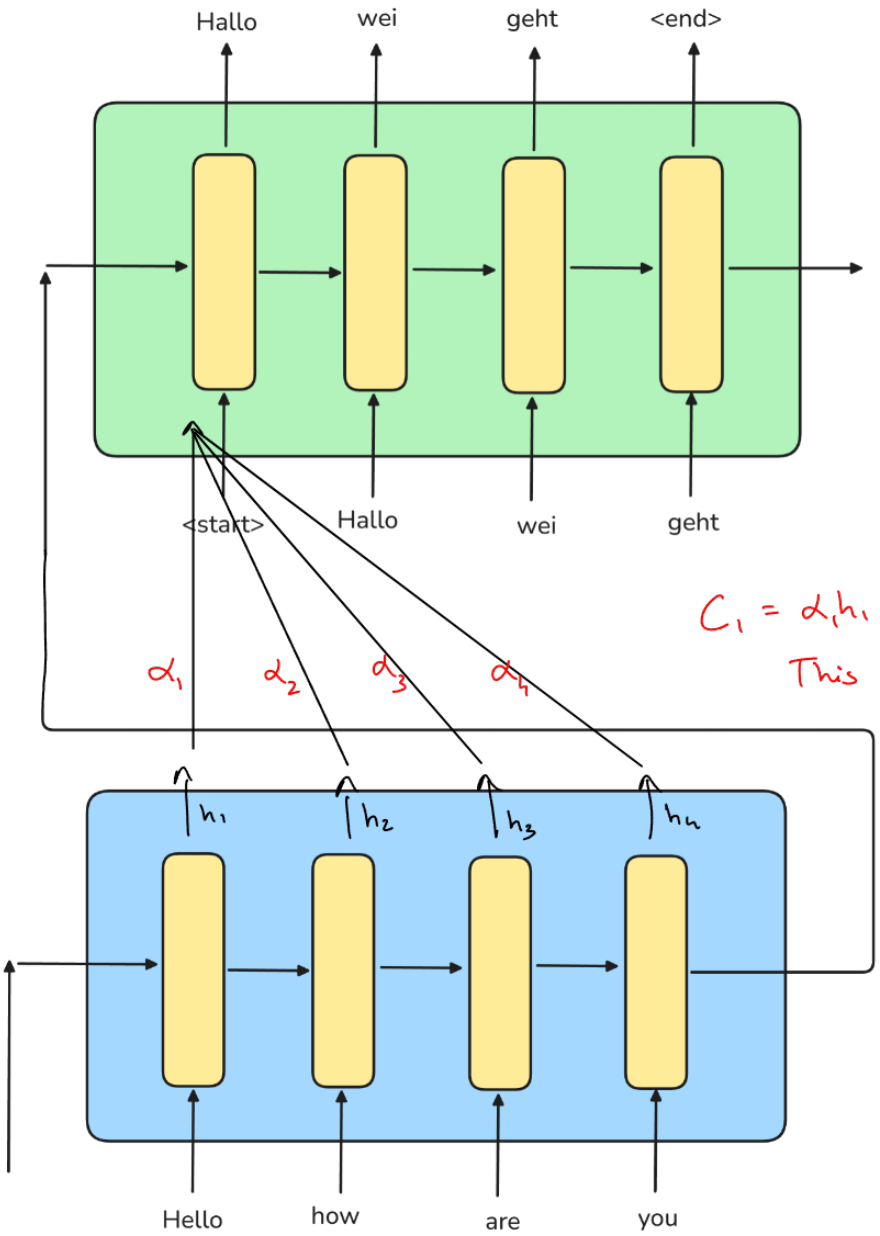
$[Y_{i-1}, S_{i-1}, C_i]$ — Attention input

Now what is Attention input (C_i)? What is it?

C_i is a vector since we know that it will have the hidden state information from encoder at timestep t.

But if we think, C_i dimension are the same as the hidden Status, but what if there are two words from the encoder that are dependent on the current word in decoder? that why we take the weighted sum.

We will assign some weight to all the hidden states from encoder to calculate the weighted sum that will output a vector called as attention input.



$C_1 = \alpha_1 h_1 + \alpha_2 h_2 + \alpha_3 h_3 + \alpha_4 h_4$

This will give us our attention input in the same dimension as hidden states

$C_1 = \alpha_1 h_1 + \alpha_2 h_2 + \alpha_3 h_3 + \alpha_4 h_4$

This will make sure that the correct required hidden state will get more weight (attention) since having more influence.

If the timestep for encoder is j and decoder is i,

$$C_i = \alpha_{ij} h_j$$

for ex.

$$C_2 = \alpha_{21} h_1 + \alpha_{22} h_2 + \alpha_{23} h_3 + \alpha_{24} h_4$$

Total there will be $i \times j = 4 \times 4 = 16$ total alphas

Now, lets focus on how to calculate those α

Each α_t depends on a total of two inputs.

i) h_i (from encoder)

ii) S_{i-1} (from decoder)

Why S from decoder?

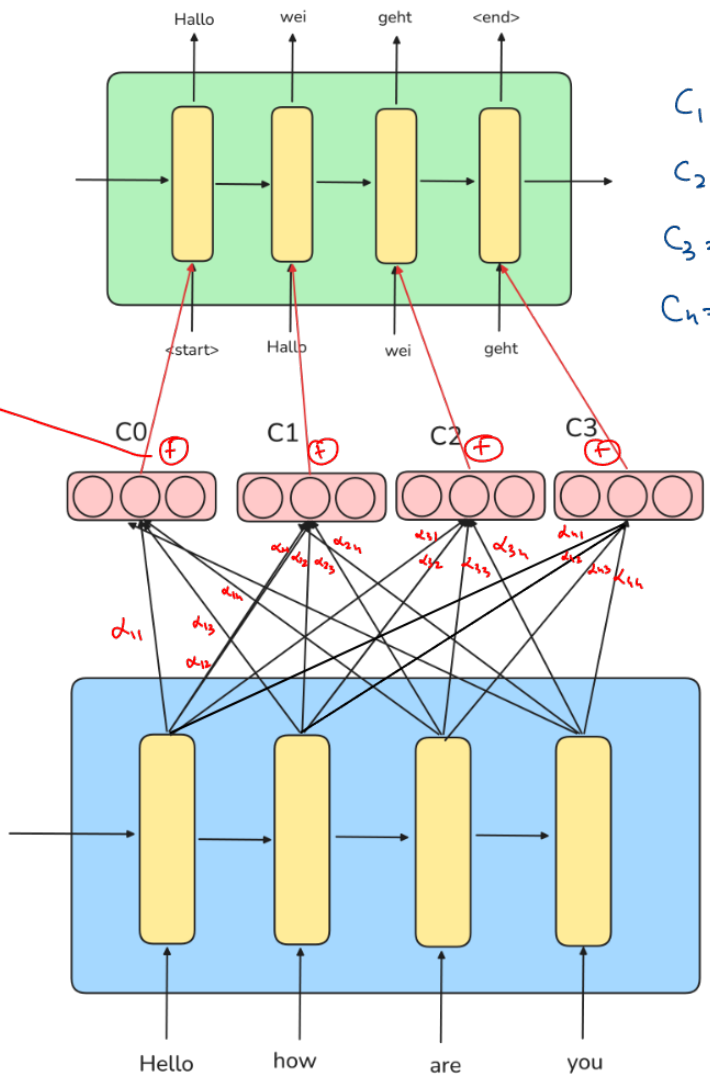
Because at each timestep, we a asking our model that "given all the translation done till now after that, what will be the role of h_i in predicting the new output.

So, $\alpha_{ij} = f(h_j, S_{i-1})$

It is simply a function with two inputs : h_j, S_{i-1}

Whats the function?

Instead of deciding the function, researchers simply used an ANN because ANN when given enough data will behave as "Universal function Approximator"



$C_1 = \alpha_{11}h_1 + \alpha_{12}h_2 + \alpha_{13}h_3 + \alpha_{14}h_4$

$C_2 = \alpha_{21}h_1 + \alpha_{22}h_2 + \alpha_{23}h_3 + \alpha_{24}h_4$

$C_3 = \alpha_{31}h_1 + \alpha_{32}h_2 + \alpha_{33}h_3 + \alpha_{34}h_4$

$C_4 = \alpha_{41}h_1 + \alpha_{42}h_2 + \alpha_{43}h_3 + \alpha_{44}h_4$

Weighted sum to find C_0 (Same for all

Remember C is Context vector

Here we are sending different context vector at each timestep.

Therefore in summary, in vanilla enc-dec architecture we gave two inputs $[h_{i-1}, s_{i-1}]$ but in attention mechanism introduction in architecture, we have inputs $[h_{i-1}, s_{i-1}, c_i]$ we introduced c which is attention input

c is simply a context vector that is calculated by weighted sum.

$c_i = \alpha_{ij} h_i$ (α values also called as Alignment Scores)

where α_{ij} is a ANN and h is simply hidden states at those timestamp.

The introduction of Attention Mechanism had significant improvements.

Bahdanau Attention Vs Luong Attention

also called as: (Additive Attention) (Multiplicative attention)

The main difference comes in how we calculate the α values.

So, till now, here's the pipeline

Encoder $\longrightarrow$ decoder $\longrightarrow$ 3 inputs $\begin{cases} s_i \\ h_{i-1} \\ c_i \longrightarrow \alpha_{ij} h_j \end{cases}$

$\alpha_{ij} h_j$ ← FCC ↓ softmax ↓ Attention Vector

Now, in Bahdanau Attention, Instead of manually defining function, we approximate it using a Feed forward Neural Network (previous one)

(What we were talking earlier is Bahdanau's attention)

Now, we will simply go deeper into the architecture and understand how the Neural Network is implemented in order to find the α values (alignment values)

lets say we are at timestep 1, here the decoder's cell will require three inputs. S_0, h_0, C_1

Now, we have S_0 and h_0 but to find C_1 :

$$C_1 = \alpha_{11} h_1 + \alpha_{12} h_2 + \alpha_{13} h_3 + \alpha_{14} h_4$$

(generalized: $C_{ij} = \sum \alpha_{ij} h_j$)

Now we have to find value of α $(\alpha_{11}, \alpha_{12}, \alpha_{13}, \alpha_{14})$

$$\alpha_{ij} = f(h_j, S_{i-1})$$

h_j: from encoder

S_{i-1}: from decoder (this is 4D)

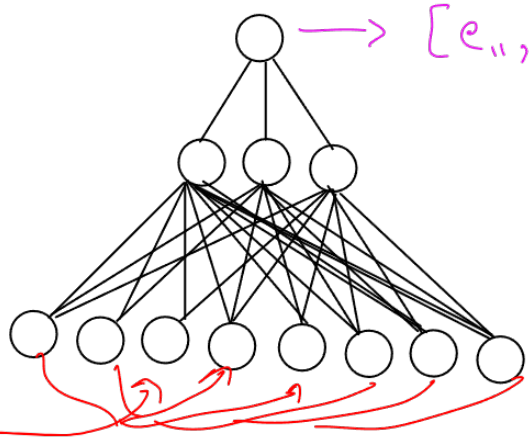
$S_0 = [S_{01}, S_{02}, S_{03}, S_{04}]$

We now concetenate S_0 vector with all values of h (h_0, h_1, h_2, h_3)

This will be simply a matrix

$$\begin{bmatrix} S_{01} & S_{02} & S_{03} & S_{04} & h_{11} & h_{12} & h_{13} & h_{14} \\ S_{01} & S_{02} & S_{03} & S_{04} & h_{21} & h_{22} & h_{23} & h_{24} \\ S_{01} & S_{02} & S_{03} & S_{04} & h_{31} & h_{32} & h_{33} & h_{34} \\ S_{01} & S_{02} & S_{03} & S_{04} & h_{41} & h_{42} & h_{43} & h_{44} \end{bmatrix}$$

Now we will feed each row one by one to the Feed forward Neural Network (or we can also send entire matrix at once (batch operation))



$\rightarrow [e_{11}, e_{12}, e_{13}, e_{14}]$

$\downarrow$

$\boxed{\text{softmax}}$ (Normalizing)

$f = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$

$\downarrow$

$[\alpha_{11}, \alpha_{12}, \alpha_{13}, \alpha_{14}]$

$\downarrow$

Context vector

Now for timestep $t = 2$

Encoder $\longrightarrow$ Decoder $\longrightarrow$ $\boxed{t = 2}$

3 inputs

$S_1 \quad h_1 \quad C_2$

$$C_2 = \alpha_{21} h_1 + \alpha_{22} h_2 + \alpha_{23} h_3 + \alpha_{24} h_4$$

$$\alpha_{ij} = f(h_j, s_{i-1})$$

h_j: 4D, s_{i-1}: 4D

$\oplus$ Concetanate

$$\begin{bmatrix} S_{11} & S_{12} & S_{13} & S_{14} & h_{11} & h_{12} & h_{13} & h_{14} \\ S_{21} & S_{22} & S_{23} & S_{24} & h_{21} & h_{22} & h_{23} & h_{24} \\ S_{31} & S_{32} & S_{33} & S_{34} & h_{31} & h_{32} & h_{33} & h_{34} \\ S_{41} & S_{42} & S_{43} & S_{44} & h_{41} & h_{42} & h_{43} & h_{44} \end{bmatrix}$$

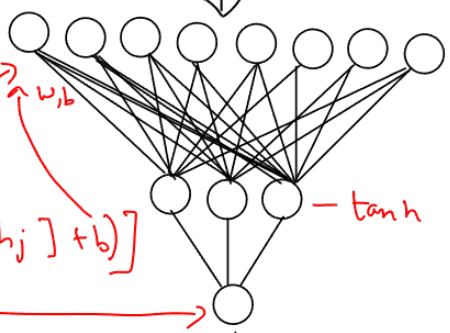
$$C_{ij} = \sum \alpha_{ij} h_j$$

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{n} \exp(e_{ij})}$$

$$e_{ij} = V\left[\tanh\left(W[S_{i-1} : h_j] + b\right)\right]$$

W, b

weight at this layer



— tanh Activation functions

$[e_{21}, e_{22}, e_{23}, e_{24}]$

$\boxed{\text{Softmax}}$

$[\alpha_{21}, \alpha_{22}, \alpha_{23}, \alpha_{24}]$

$$C_2 = \alpha_{21} h_1 + \alpha_{22} h_2 + \alpha_{23} h_3 + \alpha_{24} h_4$$

$C_2 \quad S_1 \quad h_1$

h_t

Luong Attention

There are a total of 2 differences in Luong attention compared to bahdanau attention.

difference ① Instead of using S_{t-1} (previous hidden state of decoder)

Luong attention uses the current hidden state of decoder to compute attention.

This means the decoder RNN runs first to produce S_t and then attention is computed using S_t.

The context vector C_t is then concatenated with S_t and passed through a linear layer (softmax) to produce the output.

Order of operation flips :

Bahdanua : attention $\longrightarrow$ decoder RNN

Luong : decoder RNN $\longrightarrow$ attention

Difference ② : How Score is Computed (α)

Instead of applying :

$$e_{ij} = V^T \tanh(W[S_{t-1} : h_t] + b)$$

through a FCC,

we simply use dot product

(we use current timestep just to get more updated info)

$$\alpha_{ij} = f(S_i, h_j)$$

(note that this is changed hidden state for current timestep)

Here we simply do dot product of S_i, h_j $\longrightarrow$ scalar value (e_{ij})

$$[a \ b \ c \ d] \begin{bmatrix} e \\ f \\ g \\ h \end{bmatrix} \longrightarrow ae + bf + cg + dh = \text{scalar}$$

same for other α_{ij}

Softmax

α_{ij} value

Architecture

timestep $t = 1$

Encoder $\longrightarrow$ Decoder $\longrightarrow$ $\boxed{t = 1}$

3 inputs

$S_1 \quad h_1 \quad C_2$

$$C_2 = \alpha_{21} h_1 + \alpha_{22} h_2 + \alpha_{23} h_3 + \alpha_{24} h_4$$

$$\alpha_{ij} = f(h_j, s_i)$$

4D $\odot$ 4D

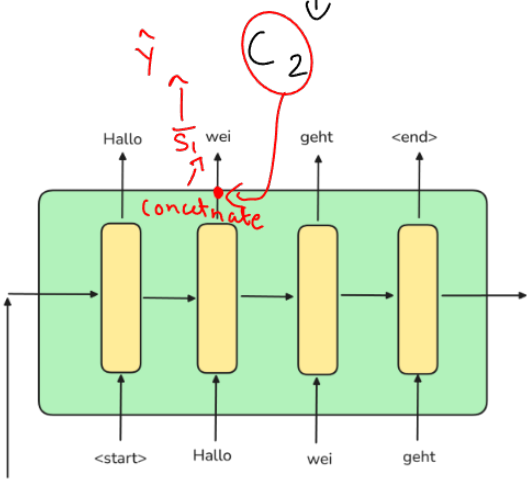
dot product

$$\underbrace{s_1 h_1 \quad s_2 h_2 \quad s_3 h_3 \quad s_4 h_4}$$

$$[e_{11} \quad e_{12} \quad e_{13} \quad e_{14}]$$

$\boxed{\text{Softmax}}$

$$[\alpha_{11} \quad \alpha_{12} \quad \alpha_{13} \quad \alpha_{14}]$$

$$C_2 = \alpha_{21} h_1 + \alpha_{22} h_2 + \alpha_{23} h_3 + \alpha_{24} h_4$$

Here main difference is that the context vector goes into the output instead of input